18. Spark MLlib

Overview

Apache Spark scales feature engineering and MLlib pipelines across partitions. This module covers DataFrame APIs, train/test splits at scale, MLlib estimators/transformers, and Windows-specific Spark setup used in the lab.

Lab: Lab 18.

Contents

Module track

This module starts a new track. Later modules refer back using **Module NN** labels.

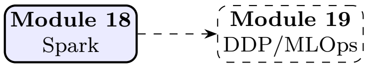

Figure 1: Module sequence (solid box: this PDF; dashed: typical next step).

1 Distributed ML Motivation

Single-machine training limits data and model scale. Apache Spark MLlib brings distributed ML to cluster compute, complementing neural training in **Module 19**.

2 Spark Data Model

Resilient Distributed Datasets (RDDs) and DataFrames partition data across workers:

$$\mathcal{D} = \bigcup_{p=1}^{P} \mathcal{D}_p, \quad \text{processed in parallel map/reduce stages.} \tag{1}$$

Lazy evaluation builds DAG of transformations; actions trigger execution.

Note. Data locality: "move computation to data" minimizes network shuffle, critical for terabyte logs.

3 MLlib Pipeline API

Estimator → Transformer → Model workflow:

1. `StringIndexer`, `VectorAssembler` (feature prep)
2. `StandardScaler`, `PCA`
3. `LogisticRegression`, `RandomForest`, `GBT`
4. `CrossValidator` for hyperparameter search

4 Example: Distributed Logistic Regression

Minimize $\mathcal{L}(\mathbf{w}) = \sum_{i\in\mathcal{D}} \ell(y_i, \mathbf{w}^\top \mathbf{x}_i) + \lambda\|\mathbf{w}\|^2$ via gradient descent:

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta_t \sum_{p=1}^{P} \nabla_{\mathbf{w}} \mathcal{L}_p(\mathbf{w}_t). \tag{2}$$

Spark aggregates partial gradients on driver each iteration.

5 MLlib vs. Deep Learning

6 Hyperparameter Tuning

`CrossValidator` with `ParamGridBuilder` runs parallel fits:

$$\theta^* = \arg\max_{\theta\in\Theta} \frac{1}{K} \sum_{k=1}^{K} \mathrm{Score}_k(\theta). \tag{3}$$

Table 1: When to use Spark MLlib.

Spark MLlib strength	**Prefer PyTorch/TF**
Tabular ML at TB scale	CNN/Transformer on GPU
Feature engineering pipelines	End-to-end backprop
SQL + ML unified	Research prototyping

7 Cluster Configuration

Set `spark.executor.memory`, `spark.sql.shuffle.partitions` $\approx$ 2–3$\times$ cores. Windows dev uses local[*] master; production uses YARN/K8s.

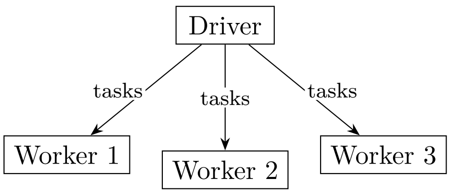

Figure 2: Spark driver coordinates partitioned tasks.

8 Lab

Lab 18 runs MLlib classification on a distributed dataset; tune λ in (4). Scale neural training with DDP in **Module 19**.

9 Feature Stores and Pipelines

Offline features materialized to parquet on HDFS feed both training and serving:

$$\mathbf{x}_i = \phi(\text{raw}_i), \quad \text{same } \phi \text{ in batch and online to avoid skew.} \tag{4}$$

Spark Structured Streaming applies micro-batch ML inference on live events.

9.1 MLlib vs. Spark ML

`spark.ml` DataFrame API supersedes RDD-based `spark.mllib` for new projects.

10 Extended Intuition

Spark distributes data across partitions so transforms and ML training run in parallel on clusters (or locally with reduced cores). MLlib pipelines chain `Transformer` stages (feature extraction) and an `Estimator` (model fit) with consistent API. DataFrames are lazy: transformations build a DAG executed on `action` calls like `count`, `collect`, or `fit`.

At scale you avoid `collect()` on large datasets; use `agg`, `groupBy`, or write parquet to disk instead. Train/test splits should be reproducible with `seed` on `randomSplit`. Hyperparameters (e.g., regParam in logistic regression) interact with feature scaling; `StandardScaler` as pipeline stage prevents leakage if fit only on train.

Windows setup quirks (Hadoop winutils, JAVA_HOME) are covered in the repo setup guide; local mode with `master("local[*]")` suffices for labs. Distributed training in **Module 19** moves neural workloads to DDP; Spark handles tabular ETL and classical ML at scale.

11 Numerical Walkthrough

Binary classification on 1M rows, 20 numeric features, 8 Spark partitions $\Rightarrow \approx 125k$ rows per partition.

Logistic regression with regParam $\lambda = 0.01$, elasticNetParam 0.0 (pure L2):

$$\min_w \frac{1}{N} \sum_i \log(1 + e^{-y_i w^\top x_i}) + \lambda \|w\|_2^2. \tag{5}$$

`LogisticRegression(maxIter=100, regParam=0.01)` on local[4]: fit might take 30 to 90 seconds depending on disk and CPU.

Metrics from `MulticlassClassificationEvaluator` with `metricName="areaUnderROC"`: AUC 0.82 to 0.88 plausible on clean synthetic data. Cache dataframe after heavy featurization: `df.cache()` before cross-validation loops.

Pipeline stages (MLlib):

```
VectorAssembler(inputCols=features, outputCol="features")
StandardScaler(inputCol="features", outputCol="scaledFeatures")
LogisticRegression(labelCol="label", featuresCol="scaledFeatures")
```

12 PyTorch Implementation Notes

Spark labs use PySpark, not PyTorch; API patterns still matter for hybrid pipelines feeding PyTorch later.

- Create session:

  ```
  SparkSession.builder \
      .master("local[*]") \
      .appName("lab18") \
      .getOrCreate()
  ```

- Read CSV: `spark.read.option("header",True).csv(path)`; cast columns with `withColumn(..., col(...).cast("double"))`.
- Fit pipeline: `model = pipeline.fit(train_df)` with stages `[assembler, scaler, lr]`.
- `CrossValidator(estimator=pipeline, evaluator=evaluator, numFolds=3)` for simple tuning.
- Export predictions:

  ```
  model.transform(test_df).select("label", "prediction") \
      .write.mode("overwrite").parquet(out)
  ```

```
from pyspark.ml import Pipeline
from pyspark.ml.classification import LogisticRegression
from pyspark.ml.feature import VectorAssembler, StandardScaler

pipeline = Pipeline(stages=[assembler, scaler, LogisticRegression(maxIter=100)])
cv_model = cv.fit(train_df)
```

13 Local GPU Constraints

Spark MLlib on this module is CPU/RAM bound, not GPU. A laptop with 16 GB RAM: keep local partitions modest; `local[4]` not `local[*]` if memory tight on 1M+ rows.

Persist strategically; `unpersist()` when done. GPU in **Module 19** is separate from Spark executors. If converting Spark output to PyTorch tensors, batch export via parquet avoids giant `collect()`.

14 Debugging Checklist

- `Java heap space`: increase driver memory `spark.driver.memory=4g` or reduce data size.
- `winutils` error on Windows: set `HADOOP_HOME` per setup script in repo.
- AUC 0.5: labels not binary encoded, or features constant after bad assembler input cols.
- Slow fit: data skew on one partition; repartition `df.repartition(16)`.
- Leakage: scaler fit on full dataset before split; use Pipeline inside CrossValidator only on train.

14.1 Partitions, Skew, and Hybrid Spark-to-PyTorch Handoff

Spark splits DataFrames into partitions; each partition is processed independently on an executor thread during `map` or ML fit tasks [3].

Default partition count follows `spark.sql.shuffle.partitions` (often 200). For small datasets this creates many tiny tasks; for very large datasets too few partitions can cause executor OOM. Spark documentation recommends sizing partitions so each task handles on the order of 128 MB of data after serialization; adjust with `df.repartition(N)` after heavy filters.

Skew happens when one key dominates (fraud detection with 99% negatives): a single partition holds most rows and becomes the straggler at every shuffle. Salting or two-phase aggregation (partial aggregate per salted key, then final merge) fixes hot keys without rewriting the whole pipeline.

`cache()` materializes partitions in memory (spill to disk if RAM is tight); use before iterative ML or cross-validation, then `unpersist()` when done.

Broadcast variables ship a small lookup table to every executor once; join a small dictionary to a large table via broadcast hash join instead of a shuffle join when the lookup fits in memory.

UDFs (`@pandas_udf` or Python UDF) are flexible but slower than built-in `spark.sql.functions`; prefer vectorized SQL expressions when possible.

Feature stores often write curated parquet with stable schemas; downstream PyTorch `Dataset` classes read shard files directly, avoiding `collect()` on large frames.

MLlib `PipelineModel` saved with `write().overwrite().save(path)` bundles assembler, scaler, and classifier stages for batch scoring on fresh data. Load with `PipelineModel.load(path)` in a scheduled job; log model version and training data hash alongside metrics for audit trails.

When the same features feed neural training in **Module 19**, export scaled feature columns to parquet with an explicit train/val split column rather than re-splitting in PyTorch and leaking distribution shift.

Monitor executor UI during first fit: if one task takes $10\times$ longer than others, investigate partition skew before tuning the algorithm. One-hot encoding high-cardinality categoricals inside Spark without target encoding leaks if fit on full data; use `StringIndexer` + `OneHotEncoder` inside Pipeline on train split only. Spark ML `LinearRegression` and `GBTClassifier` share the same Pipeline pattern; swap estimator stage without rewriting featurization when prototyping models. Writing parquet with `compression="snappy"` balances read speed and disk size for handoff to PyTorch; `zstd` saves more space at higher CPU cost on read. Structured streaming (`readStream`) extends the same DataFrame API to live logs; batch labs use static CSV but production scoring often wraps `PipelineModel.transform` in a foreachBatch sink. Hyperparameter grid with `CrossValidator`: 3 folds $\times$ 3 regParam values $\times$ 100 maxIter fit = 900 sub-fits; cache featurized train before grid search or each fold recomputes assembly. Spark UI `Stages` tab shows shuffle read/write bytes; shuffle-heavy joins before ML fit dominate wall time more often than the solver itself. Convert Spark ML predictions to pandas only for plots on $\leq 10k$ rows; use `summary()` and `agg` for metrics at full scale. MLlib `VectorAssembler` rejects nulls; impute or drop before assembly or fit throws opaque `IllegalArgumentException` on first action.

15 Practice Problems

Problem 1. Estimate rows per partition for 2M rows and 16 partitions after repartition.

Problem 2. Grid search `regParam` over $\{0.001, 0.01, 0.1\}$ with 3-fold CV; report best AUC.

Problem 3. Compare training time `local[1]` vs `local[4]` on same dataset.

Problem 4. Write train predictions to parquet and load a 10k-row sample into pandas for inspection.

Complete **Lab 18**; feeds operational patterns for **Module 19**; model explanations in **Module 20**.

Source certification

This module lists where each core definition, equation, figure, and summary table comes from. Pedagogical simplifications (lab batch sizes, epoch counts, illustrative plots) are labeled in extension sections and are not claimed as optimal production settings.

Table 3: Certified topic map for Module 18.

Topic	Primary source	Where in this PDF
Spark distributed model	[3]	Section 2
MLlib Pipeline API	[2]	Section 3
Partition / shuffle guidance	[3]	Extension section

Full bibliographic entries appear below. Figures are schematic unless a caption cites a specific paper figure. If a statement is not listed here, treat it as general ML practice or lab-specific guidance, not a cited theorem.

References

[1] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *ICLR*, 2015.

[2] Xiangrui Meng, Joseph Bradley, Burak Yavuz, et al. MLlib: Machine learning in Apache Spark. 2016.

[3] Matei Zaharia, Reynold S. Xin, Patrick Wendell, et al. Apache Spark: A unified engine for big data processing. 2016.